

Verification of Functional Programs

Introduction

Andrés Sicard-Ramírez

EAFIT University

Semester 2014-1

Administrative Information

Course web page

<https://asr.github.io/courses/verification-of-functional-programs/>

Evaluation

Homework 30%

Presentation 30%

Final project 40%

Preliminaries

Notation

Sometimes we write $\forall x\alpha$ or $\forall x.\alpha$ instead of $\forall x(\alpha)$. In $\forall x.\alpha$, the scope of the quantifier extends as far as possible, e.g. $\forall x.\alpha \wedge \beta$ means $\forall x(\alpha \wedge \beta)$. Similar for $\exists$.

Source code

All code in the examples have been tested with [Agda](#) 2.6.0.1, [Coq](#) 8.9.1 and [Isabelle](#) 2019 (June 2019).

U\$22.2 to U\$59.5 billion!*

*Source: Tassey (2002).

Motivational Example

‘Every functional programmer **worth his salt** knows how to reverse a list, debug the code, and prove that list reversal is its own inverse.’ (Swierstra and Altenkirch **2007**, p. 25)

Motivational Example

‘Every functional programmer **worth his salt** knows how to reverse a list, debug the code, and prove that list reversal is its own inverse.’ (Swierstra and Altenkirch 2007, p. 25)

Let’s go (**Haskell** code) ...

```
(++) :: [a] → [a] → [a]
[]      ++ ys = ys
(x : xs) ++ ys = x : (xs ++ ys)
```

```
rev :: [a] → [a]
rev []      = []
rev (x : xs) = rev xs ++ [x]
```

To prove that the rev function is an involution.

Motivational Example

Example

Proving $\text{rev } (\text{rev } xs) = xs$.

Case [].

$$\begin{aligned}\text{rev } (\text{rev } []) &= \text{rev } [] && (\text{rev.1}) \\ &= [] && (\text{rev.1})\end{aligned}$$

Motivational Example

Example

Proving $\text{rev } (\text{rev } xs) = xs$.

Case [].

$$\begin{aligned}\text{rev } (\text{rev } []) &= \text{rev } [] && (\text{rev.1}) \\ &= [] && (\text{rev.1})\end{aligned}$$

Case $x:xs$.

$$\begin{aligned}\text{rev } (\text{rev } (x : xs)) &= \text{rev } (\text{rev } xs ++ [x]) && (\text{rev.2}) \\ &= x : \text{rev } (\text{rev } xs) && (\text{auxiliary thm.}) \\ &= x : xs && (\text{IH})\end{aligned}$$

Auxiliary theorem: $\text{rev } (ys ++ [x]) = x : \text{rev } ys$.

Motivational Example

Observation

The auxiliary theorem

$$\text{rev } (ys \ ++ \ [x]) = x : \text{rev } ys$$

is a generalisation of the required result

$$\text{rev } (\text{rev } xs \ ++ \ [x]) = x : \text{rev } (\text{rev } xs).$$

‘A standard method of generalisation is to look for a sub-expression that appears on both sides of the equation and replace it by a variable.’ (Bird and Wadler 1988, p. 124)

Observations from the Motivational Example

- Inductive data types $\Rightarrow$ **Structural induction** for reasoning about them.

Observations from the Motivational Example

- Inductive data types $\Rightarrow$ **Structural induction** for reasoning about them.
- **Equational reasoning** (process of replacing like for like using the substitutivity property and the equivalence properties of the equality) based on the **referential transparency**.

Observations from the Motivational Example

- Inductive data types $\Rightarrow$ **Structural induction** for reasoning about them.
- **Equational reasoning** (process of replacing like for like using the substitutivity property and the equivalence properties of the equality) based on the **referential transparency**.
- **Generalisation** of auxiliary theorem (including the inductive hypothesis) $\Rightarrow$ Proofs by induction are difficult to automatise.

Questions from the Motivational Example

- What about $\perp$?

$$\text{rev } (\text{rev } \perp) \stackrel{?}{=} \perp$$

Questions from the Motivational Example

- What about $\perp$?

$$\text{rev } (\text{rev } \perp) \stackrel{?}{=} \perp$$

- **Extend** structural induction for handling $\perp$.

Questions from the Motivational Example

- What about $\perp$?

$$\text{rev } (\text{rev } \perp) \stackrel{?}{=} \perp$$

- **Extend** structural induction for handling $\perp$.
- Choose a programming logic to behaviours of programs on **total** and **finite** elements of data structures (Bove, Dybjer and Sicard-Ramírez 2009; Dybjer 1985).

Questions from the Motivational Example

- What about $\perp$?

$$\text{rev } (\text{rev } \perp) \stackrel{?}{=} \perp$$

- **Extend** structural induction for handling $\perp$.
- Choose a programming logic to behaviours of programs on **total** and **finite** elements of data structures (Bove, Dybjer and Sicard-Ramírez 2009; Dybjer 1985).
- '**Morally**' correct reasoning (Danielsson, J. Hughes, Jansson and Gibbons 2006).

Questions from the Motivational Example

- What about if `xs` is an infinite list?

$$\text{rev } (\text{rev } xs) \stackrel{?}{=} xs$$

Questions from the Motivational Example

- What about if `xs` is an infinite list?

$$\text{rev } (\text{rev } xs) \stackrel{?}{=} xs$$

- Co-inductive data types $\Rightarrow$ **Co-induction** for reasoning about them (Gibbons and Hutton 2005).

Questions from the Motivational Example

- What about if xs is an infinite list?

$$\text{rev } (\text{rev } xs) \stackrel{?}{=} xs$$

- Co-inductive data types $\Rightarrow$ **Co-induction** for reasoning about them (Gibbons and Hutton 2005).
- Choose a programming logic to behaviours of programs on **total** (**finite** or **potentially unbounded**) elements of data structures (Bove, Dybjer and Sicard-Ramírez 2012; Dybjer and Sander 1989).

Questions from the Motivational Example

- The rev function is $O(n^2)$. Why are we reasoning about it?

```
GHCi> rev [1..107]
```

```
*** Exception: stack overflow
```

Questions from the Motivational Example

- The `rev` function is $O(n^2)$. Why are we reasoning about it?

```
GHCi> rev [1..10^7]
```

```
*** Exception: stack overflow
```

The `reverse` function in the `Data.List` library ([GHC 7.8.2](#)) is $O(n)$:

```
reverse l = rev l []
```

where

```
rev []      a = a
```

```
rev (x:xs) a = rev xs (x:a)
```

Questions from the Motivational Example

- In relation to the formal verification of `find` or `gcd` algorithms versus the verification of **real** programs:

‘They are differences in kind. Babysitting for a sleeping child for one hour does not scale up to raising a family of ten—the problems are essentially, fundamentally different.’ (De Millo, Lipton and Perlis 1979, p. 278)

Verification of Functional Programs: Research Areas

Area	Research focuses on
Semantics definitions	Defining new concepts
Transformation rules	Programming transformations
Functional properties verification	The input and output correspondence of programs
Non-functional properties verification	Properties such as memory consumption or parallel performance

Source: Achten, van Eekelen, Koopam and Morazán (2010).

(Incomplete) Time Line

- 1949 Alan Turing (1949). Checking a Large Routine. In: Report of a Conference on High Speed Automatic Calculating Machines, pp. 67–69.
- 1957 J. W. Backus, R. J. Beeber, S. Best, R. Goldberg, L. M. Haibt, H. L. Herrick, R. A. Nelson, D. Sayre, P. B. Sheridan, H. Stern, I. Ziller, R. A. Hughes and R. Nutt (1957). The FORTRAN Automatic Coding System. In: Proceedings Western Joint Computer Conference, pp. 188–198. (FORTRAN)
- 1958 John McCarthy (1960). Recursive Functions of Symbolic Expressions and their Computation by Machine, Part I. Communications of the ACM 3.4, pp. 184–195. DOI: [10.1145/367177.367199](https://doi.org/10.1145/367177.367199). (Lisp)
- 1960 J. W. Backus, F. L. Bauer, J. Green, C. Katz, J. McCarthy, A. J. Perlis, H. Rutishauser, K. Samelson, B. Vauquois, J. H. Wegstein, A. van Wijngaarden and M. Woodger (1960). Report on the Algorithmic Language ALGOL 60. Communications of the ACM 3.5. Ed. by Peter Naur, pp. 299–314. DOI: [10.1145/367236.367262](https://doi.org/10.1145/367236.367262). (ALGOL 60)

(Incomplete) Time Line

- 1961 John McCarthy (1961). A Basis for a Mathematical Theory of Computation. In: Proceedings Western Joint Computer Conference, pp. 225–238.
- 1966 Peter Naur (1966). Proof of Algorithms by General Snapshots. BIT 6.4, pp. 310–316.
- 1967 Robert W. Floyd (1967). Assigning Meanings to Programs. In: Mathematical Aspects of Computer Science. Ed. by Jacob T. Schwartz. Vol. 19. Proceedings of Symposia in Applied Mathematics, pp. 19–32.
- 1968 ‘In 1968, a NATO Conference on Software Engineering was held in Garmisch, Germany, ...For the **first time**, a consensus emerged that there really was a software **crisis**, that programming was not very well understood.’ (Gries 1981, p. 296)
- 1969 C. A. R. Hoare (1969). An Axiomatic Basis for Computer Programming. Communications of the ACM 12.10, 576–580(3). DOI: [10.1145/363235.363259](https://doi.org/10.1145/363235.363259).

(Incomplete) Time Line

- 1971 Per Martin-Löf (1971). A Theory of Types. Tech. rep. University of Stockholm.
- 1973 Per Martin-Löf (1975). About Models for Intuitionistic Type Theories and the Notion of Definitional Equality. In: Proceedings of the Third Scandinavian Logic Symposium. Ed. by Stig Kanger. Vol. 82. Studies in Logic and the Foundations of Mathematics. Elsevier, pp. 81–109.
- 1979 Per Martin-Löf (1982). Constructive Mathematics and Computer Programming. In: Logic, Methodology and Philosophy of Science VI (1979). Ed. by L. J. Cohen, J. Los, H. Pfeiffer and K.-P. Podewski. Vol. 104. Studies in Logic and the Foundations of Mathematics. North-Holland Publishing Company, pp. 153–175. DOI: [10.1016/S0049-237X\(09\)70189-2](https://doi.org/10.1016/S0049-237X(09)70189-2).
- 1981 Bengt Nordström (1981). Programming in Constructive Set Theory: Some Examples. In: Proceedings of the 1981 Conference on Functional Programming Languages and Computer Architecture (FPCA 1981). ACM, pp. 141–154.

References

- Peter Achten, Marko van Eekelen, Pieter Koopam and Marco T. Morazán (2010). Trends in *Trends in Functional Programming* 1999/2000 versus 2007/2008. *Higher-Order Symbolic Computation* 23.4, pp. 465–487. DOI: [10.1007/s10990-011-9074-z](https://doi.org/10.1007/s10990-011-9074-z) (cit. on p. 23).
- J. W. Backus, F. L. Bauer, J. Green, C. Katz, J. McCarthy, A. J. Perlis, H. Rutishauser, K. Samelson, B. Vauquois, J. H. Wegstein, A. van Wijngaarden and M. Woodger (1960). Report on the Algorithmic Language ALGOL 60. *Communications of the ACM* 3.5. Ed. by Peter Naur, pp. 299–314. DOI: [10.1145/367236.367262](https://doi.org/10.1145/367236.367262) (cit. on p. 24).
- J. W. Backus, R. J. Beeber, S. Best, R. Goldberg, L. M. Haibt, H. L. Herrick, R. A. Nelson, D. Sayre, P. B. Sheridan, H. Stern, I. Ziller, R. A. Hughes and R. Nutt (1957). The FORTRAN Automatic Coding System. In: *Proceedings Western Joint Computer Conference*, pp. 188–198 (cit. on p. 24).
- Richard Bird and Philip Wadler (1988). *Introduction to Functional Programming*. Series in Computer Sciences. Prentice Hall International (cit. on p. 9).
- Ana Bove, Peter Dybjer and Andrés Sicard-Ramírez (2009). Embedding a Logical Theory of Constructions in Agda. In: *Proceedings of the 3rd Workshop on Programming Languages Meets Program Verification (PLPV 2009)*, pp. 59–66 (cit. on pp. 13–16).

References

- Ana Bove, Peter Dybjer and Andrés Sicard-Ramírez (2012). Combining Interactive and Automatic Reasoning in First Order Theories of Functional Programs. In: Foundations of Software Science and Computation Structures (FoSSaCS 2012). Ed. by Lars Birkedal. Vol. 7213. Lecture Notes in Computer Science. Springer, pp. 104–118 (cit. on pp. 17–19).
- Nils Anders Danielsson, John Hughes, Patrik Jansson and Jeremy Gibbons (2006). Fast and Loose Reasoning is Morally Correct. In: Proceedings of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2006), pp. 206–217. DOI: [10.1145/1111037.1111056](https://doi.org/10.1145/1111037.1111056) (cit. on pp. 13–16).
- Richard A. De Millo, Richard J. Lipton and Alan J. Perlis (1979). Social Processes and Proofs of Theorems and Programs. Communications of the ACM 22.5, pp. 271–280. DOI: [10.1145/359104.359106](https://doi.org/10.1145/359104.359106) (cit. on p. 22).
- Peter Dybjer (1985). Program Verification in a Logical Theory of Constructions. In: Functional Programming Languages and Computer Architecture. Ed. by Jean-Pierre Jouannaud. Vol. 201. Lecture Notes in Computer Science. Springer, pp. 334–349. DOI: [10.1007/3-540-15975-4_46](https://doi.org/10.1007/3-540-15975-4_46) (cit. on pp. 13–16).

References

- Peter Dybjer and Herbert P. Sander (1989). A Functional Programming Approach to the Specification and Verification of Concurrent Systems. *Formal Aspects of Computing* 1, pp. 303–319 (cit. on pp. 17–19).
- Robert W. Floyd (1967). Assigning Meanings to Programs. In: *Mathematical Aspects of Computer Science*. Ed. by Jacob T. Schwartz. Vol. 19. *Proceedings of Symposia in Applied Mathematics*, pp. 19–32 (cit. on p. 25).
- Jeremy Gibbons and Graham Hutton (2005). Proof Methods for Corecursive Programs. *Fundamenta Informaticae* XX, pp. 1–14 (cit. on pp. 17–19).
- David Gries (1981). *The Science of Programming*. *Monographs in Computer Science*. Springer-Verlag. DOI: [10.1007/978-1-4612-5983-1](https://doi.org/10.1007/978-1-4612-5983-1) (cit. on p. 25).
- C. A. R. Hoare (1969). An Axiomatic Basis for Computer Programming. *Communications of the ACM* 12.10, 576–580(3). DOI: [10.1145/363235.363259](https://doi.org/10.1145/363235.363259) (cit. on p. 25).
- Per Martin-Löf (1971). A Theory of Types. Tech. rep. University of Stockholm (cit. on p. 26).
- (1975). About Models for Intuitionistic Type Theories and the Notion of Definitional Equality. In: *Proceedings of the Third Scandinavian Logic Symposium*. Ed. by Stig Kanger. Vol. 82. *Studies in Logic and the Foundations of Mathematics*. Elsevier, pp. 81–109 (cit. on p. 26).

References

- Per Martin-Löf (1982). Constructive Mathematics and Computer Programming. In: Logic, Methodology and Philosophy of Science VI (1979). Ed. by L. J. Cohen, J. Los, H. Pfeiffer and K.-P. Podewski. Vol. 104. Studies in Logic and the Foundations of Mathematics. North-Holland Publishing Company, pp. 153–175. DOI: [10.1016/S0049-237X\(09\)70189-2](https://doi.org/10.1016/S0049-237X(09)70189-2) (cit. on p. 26).
- John McCarthy (1960). Recursive Functions of Symbolic Expressions and their Computation by Machine, Part I. Communications of the ACM 3.4, pp. 184–195. DOI: [10.1145/367177.367199](https://doi.org/10.1145/367177.367199) (cit. on p. 24).
- (1961). A Basis for a Mathematical Theory of Computation. In: Proceedings Western Joint Computer Conference, pp. 225–238 (cit. on p. 25).
- Peter Naur (1966). Proof of Algorithms by General Snapshots. BIT 6.4, pp. 310–316 (cit. on p. 25).
- Bengt Nordström (1981). Programming in Constructive Set Theory: Some Examples. In: Proceedings of the 1981 Conference on Functional Programming Languages and Computer Architecture (FPCA 1981). ACM, pp. 141–154 (cit. on p. 26).
- Wouter Swierstra and Thorsten Altenkirch (2007). Beauty in the Beast. A Functional Semantics for the Awkward Squad. In: Proceedings of the ACM SIGPLAN 2007 Haskell Workshop, pp. 25–36 (cit. on pp. 5, 6).

References

- Gregory Tassej (2002). The Economic Impacts of Inadequate Infrastructure for Software Testing. Tech. rep. National Institute of Standards and Technology. US Department of Commerce (cit. on p. 4).
- Alan Turing (1949). Checking a Large Routine. In: Report of a Conference on High Speed Automatic Calculating Machines, pp. 67–69 (cit. on p. 24).